

ClickHouse를 활용한 광고 데이터의 신선도와 정확도를 동시에 잡기

Confluent → S3 → ClickHouse Cloud로 구현하는
DARO 광고 데이터 파이프라인 실전 사례

PRESENTER

Andy Kim · DelightRoom Foundation Team



Alarmy

Give everyone a successful morning.

모두의 아침을 성공적으로 바꿉니다.

앱 다운로드

1위

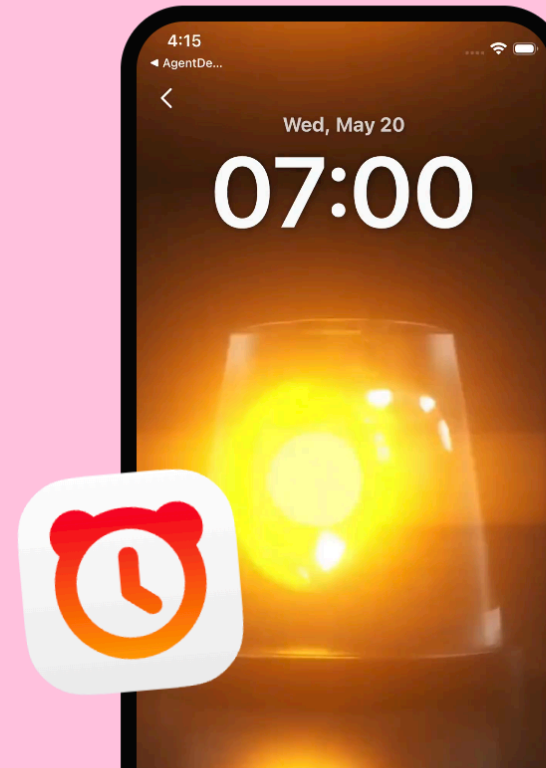
100개국에서

1억

누적 다운로드

400만

일간 활성 사용자



Daro

You focus on product. We focus on revenue.

제품에 집중하세요. 광고 수익은 우리가 책임집니다.

DARO 바로가기

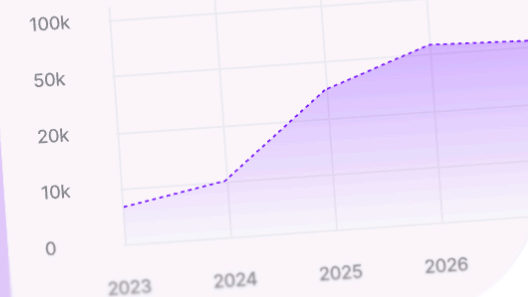
5,000만

월간 활성 사용자 수

200+

파트너사 수

Growth



DelightHub

Turning great products into lasting businesses.

좋은 제품을 오래가는 사업으로 키웁니다.

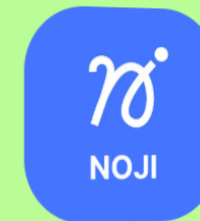
DelightHub 바로가기

6

인수한 서비스

30+

투자한 회사



4명의 엔지니어, 3개의 사업부

03 / 15

ALARMY

1억 누적 다운로드 · 100개국 1위

DARO

5,000만 MAU · 200+ 파트너사

DELIGHTHUB

6개 인수 · 30+ 투자사

DevOps

SRE

Data Platform

AX

넓은 스코프, 작은 팀

관리 영역

AWS / EKS

Compute · Storage · Network

GCP

Data · AI Platform

GWS

Google Workspace

Confluent

Kafka · Schema

ClickHouse

Cloud

AI Host Machines

In-house Infra



4명

Foundation Engineers

핵심 챌린지

비용, 운영 리소스를 효율적으로 넓은 스코프를 커버하기 = **TCO 최소화**

신선도 + 정확도의 요구사항

신선도 (FRESHNESS)

새로운 광고 지면 추가

새 퍼블리셔가 온보딩되면 즉시 수익 데이터를 확인할 수 있어야 합니다.

광고 지면 실험

A/B 실험 중 실시간에 가까운 성과 피드백이 필요합니다.



정확도 (ACCURACY)



정확한 재집계

확정된 값으로 정확히 일 단위로 교체가 되어야 합니다.



다운타임

고객 입장에서 데이터 다운타임이 생기면 안됩니다.

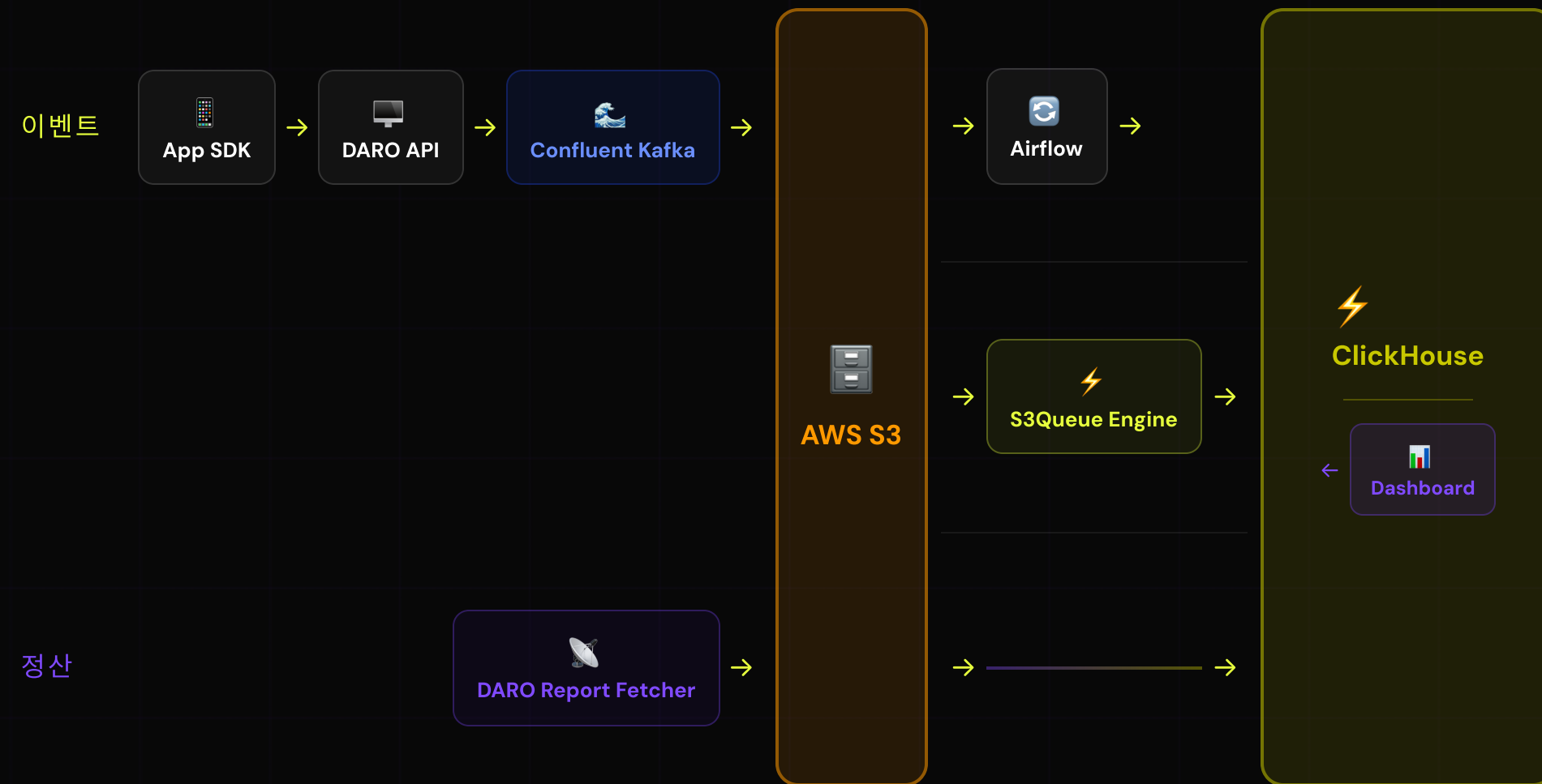
SECTION 01

ClickHouse를 선택하게 된 이유

신선도와 정확도, 두 가지를 동시에 달성하는 법

DARO 데이터 파이프라인

07 / 15



신선도: S3 Batch → S3Queue Engine

항목	S3 → AIRFLOW 배치	S3QUEUE ENGINE
동작 방식	Airflow DAG가 S3에서 읽어 ClickHouse에 적재	ClickHouse가 S3를 직접 폴링하여 자동 적재
데이터 지연	H+2 / H+3 (1시간 파티션 단위)	H+0.x (수십 분 이내)
운영 복잡도	Airflow DAG 유지보수 필요	ClickHouse 설정만으로 완결
정확도	높음 — 배치 완결 후 적재	근사치 — 불완전한 시간 파티션 포함 가능
공통점	✓ 동일한 S3 경로에서 독립적으로 읽음 — 병행 운영 가능	

S3 → AIRFLOW 배치 예시

```
-- Airflow DAG에서 실행
INSERT INTO stg_ssp__impression
SELECT * FROM s3(
  's3://daro-prod-private/topics/
  daro.ssp.impression.v1/{{partition}}/**',
  'JSONEachRow'
);
```

S3QUEUE ENGINE 설정 예시

```
CREATE TABLE stg_ssp_impression
ENGINE = S3Queue(
  's3://daro-prod-private/topics/
  daro.ssp.impression.v1/**',
  'JSONEachRow'
) SETTINGS
mode = 'unordered';
```


| 정확도: REPLACE PARTITION

09 / 15

ATOMIC

IDEMPOTENT

ZERO DOWNTIME

REPLACE PARTITION

```
ALTER TABLE table2 [ON CLUSTER cluster] REPLACE PARTITION partition_expr FROM table1
```

이 쿼리는 `table1` 의 데이터 파티션을 `table2` 로 복사하고, `table2` 에 존재하던 파티션을 대체합니다. 이 작업은 원자적으로 수행됩니다.

다음 사항에 유의해야 합니다:

- `table1` 의 데이터는 삭제되지 않습니다.
- `table1` 은 임시 테이블일 수 있습니다.

쿼리가 성공적으로 실행되려면 다음 조건을 만족해야 합니다:

- 두 테이블은 동일한 구조를 가져야 합니다.
- 두 테이블은 동일한 파티션 키, 동일한 ORDER BY 키, 동일한 기본 키를 가져야 합니다.
- 두 테이블은 동일한 스토리지 정책을 가져야 합니다.
- 대상 테이블에는 소스 테이블의 모든 인덱스와 프로젝션이 포함되어야 합니다. 대상 테이블에서 `enforce_index_structure_match_on_partition_manipulation` 설정이 활성화된 경우, 인덱스와 프로젝션은 완전히 동일해야 합니다. 그렇지 않으면 대상 테이블은 소스 테이블보다 더 많은 인덱스와 프로젝션을 포함할 수 있습니다.

하나의 테이블로 신선도와 정확도 만족시키기

10 / 15

AGG_SSP_IMPRESSION · AGGREGATINGMERGETREE

실시간 경로 — MATERIALIZED VIEW

```
CREATE TABLE agg_ssp_impression (  
  event_date      Date,  
  ad_unit_id      String,  
  bidder_name     String,  
  bid_price_sum   AggregateFunction(sum, Float64),  
  unique_devices  AggregateFunction(uniq, String)  
) ENGINE = AggregatingMergeTree()  
PARTITION BY event_date  
ORDER BY (  
  event_date,  
  ad_unit_id,  
  bidder_name  
)
```

```
CREATE MATERIALIZED VIEW mv_ssp_impression_agg  
TO agg_ssp_impression AS  
SELECT  
  event_date, ad_unit_id, bidder_name,  
  sumState(bid_price) AS bid_price_sum,    -- AggregateFunction blob  
  uniqState(device_id) AS unique_devices  
FROM stg_ssp__impression  
GROUP BY event_date, ad_unit_id, bidder_name;
```

일별 배치 — 확정 데이터로 파티션 교체

```
-- 1. 확정 데이터를 임시 테이블에 -State 형식으로 적재  
INSERT INTO agg_ssp_impression_tmp  
SELECT  
  event_date, ad_unit_id, bidder_name,  
  sumState(bid_price),  uniqState(device_id)  
FROM finalized_s3_source  
WHERE event_date = yesterday();  
  
-- 2. 원자적 교체 — 오늘 MV는 영향 없음  
ALTER TABLE agg_ssp_impression  
  REPLACE PARTITION toDate(yesterday())  
  FROM agg_ssp_impression_tmp;
```

- 쿼리 시 `sumMerge` / `uniqMerge`로 집계 상태를 최종화합니다.

SECTION 02

ClickHouse Cloud 레버리지해서 TCO 낮추는 몇 가지 방법

공유 스토리지 아키텍처가 만들어내는 운영 효율

ClickHouse Cloud Warehouse

2 / 15

USE CASE 01

QA Staging 환경

- 데이터 복제 없이 Prod와 동일한 데이터셋으로 QA 수행
- 스토리지 비용 **추가 없음** — 컴퓨팅 비용만 발생

USE CASE 02

ClickHouse로 Lightweight Data Lake 구축

- ClickHouse가 **Query Engine** 역할
- Analytics용 Read Replica Warehouse를 붙여 Prod 컴퓨팅 부하 분산
- Snowflake · BigQuery 같은 별도 플랫폼 없이 **TCO 절감**

daro-prod warehouse

Warehouses are automatically generated by ClickHouse Cloud when multiple services are accessing the same data.

Warehouse name

daro-prod warehouse

Tokyo (ap-northeast-1)

Release channel

Regular channel

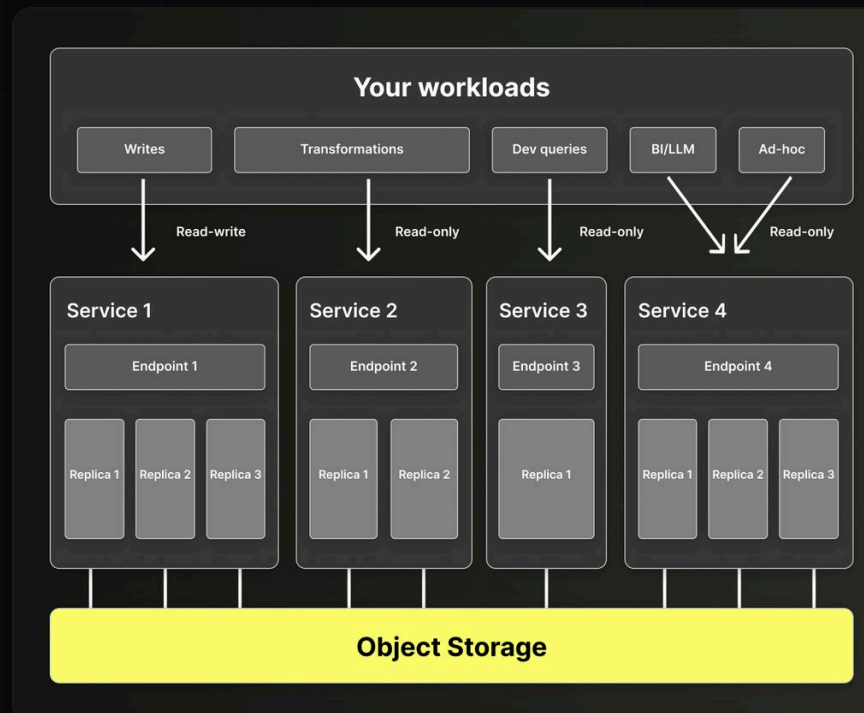
Services

Service Name	Size	Permission
daro-prod	8 ↔ 32 GiB per replica 3 replicas	Read-write
daro-prod-read-replica	8 ↔ 12 GiB per replica 2 replicas	Read-only

Cancel

Reset DB password

+ Add service



하나의 테이블에 **준실시간 스트림된 집계**와 **백필 집계**를 모두 담는게 가능합니다.

복잡한 데이터 파이프라인 없이 ClickHouse 내에서 **DDL, DML**만으로 설정이 가능합니다.

- **Materialized View**가 실시간으로 집계 상태(-State)를 누적

- **REPLACE PARTITION**이 확정 데이터로 해당 파티션을 원자적 교체

- 쿼리 시 **sumMerge / uniqMerge**로 집계 상태를 최종화

REPLACE PARTITION을 활용하면 재처리 파이프라인이 단순해집니다.

CLICKHOUSE - 3단계

```
-- 1. 임시 테이블에 적재
INSERT INTO stg_ssp__impression_tmp
SELECT * FROM s3_source
WHERE partition_key = '2024010114';

-- 2. 원자적으로 교체
ALTER TABLE stg_ssp__impression
  REPLACE PARTITION '2024010114'
  FROM stg_ssp__impression_tmp;

-- 3. 임시 테이블 삭제
DROP TABLE stg_ssp__impression_tmp;
```

AIRFLOW DAG - 멍등성 보장

```
from datetime import datetime
from airflow.decorators import dag
from airflow.providers.common.sql.operators.sql import SQLExecuteQueryOperator

@dag(schedule="@daily", start_date=datetime(2024, 1, 1), catchup=False)
def daro_ssp_daily_backfill():

    load_tmp = SQLExecuteQueryOperator(
        task_id="load_tmp", conn_id="clickhouse",
        sql="""INSERT INTO stg_ssp__impression_tmp
                SELECT * FROM s3_source
                WHERE partition_key = '{{ ds_nodash }}'""",
    )

    replace = SQLExecuteQueryOperator(
        task_id="replace_partition", conn_id="clickhouse",
        sql="""ALTER TABLE stg_ssp__impression
                REPLACE PARTITION '{{ ds_nodash }}'
                FROM stg_ssp__impression_tmp;
                DROP TABLE stg_ssp__impression_tmp""",
    )

    load_tmp >> replace

daro_ssp_daily_backfill()
```

감사합니다

질문이 있으시면 편하게 말씀해 주세요.

발표자

Andy Kim

Foundation Team · DelightRoom